

SRW3 Phase 1H — Execution-Client Integration and Consensus-Boundary Pilot

Status: PHASE 1H EXECUTED — the integration boundary is answered with a boundary-honest result: SRW3 can be cleanly embedded at the Engine-API boundary of a REAL Ethereum execution client (Geth v1.17.7, Amsterdam fork, engine_newPayloadV5 + EIP-7928 BAL) with ZERO client modification, ZERO Ethereum-validity change, and byte-identical determinism; an Ethereum-valid payload can be SRW3-invalid (demonstrated); making SRW3 rejection consensus-visible is NOT an implementation problem but a PROTOCOL problem — it requires a new fork-choice/validity rule plus identical policy distribution, and does NOT fit the existing Engine API INVALID semantics without overloading them.

This report was produced on branch phase1h, created from the Phase 1G closure commit a71ba645fa7347a422ea968db4bc96d0f4ea75b7 (tag phase1g-complete). The frozen layers were not modified: git diff over k/, python-gen/, phasele/, phase1f/, phase1g/ against the Phase 1G tip is empty. All Phase 1H code is additive (phase1h/).

PHASE 1H RESULTS

```
Selected execution client: go-ethereum (Geth) v1.17.7-stable
Client commit: 3d858f858a458effb2a563788aedf1fe65e1f0d3
(prebuilt official binary; source cloned
for the insertion-point analysis; the
in-process patch is provided but NOT BUILT
– disclosed in client/CLIENT-SELECTION.md)
Engine API spec: github.com/ethereum/execution-apis @
3b9944a07afb53a327a3b5908a607f14b4e764d8
(2026-10-07; frozen copies + SHA-256 in
spec-baseline/)
Method versions exercised: engine_newPayloadV5, engine_getPayloadV6,
engine_forkchoiceUpdatedV4,
engine_getPayloadBodiesByHashV2
Fork/configuration: private devnet, chainId 93471, ALL forks at
time 0 -> ACTIVE FORK = Amsterdam;
ExecutionPayloadV4 (blockAccessList +
slotNumber); genesis
phase1h/devnet/genesis.json
```

Exact integration point.

```
`eth/catalyst/api.go :: ConsensusAPI.NewPayloadV5 (line 862) →
api.newPayload (line 905) →
Blockchain().InsertBlockWithoutSetHead (core/blockchain.go:2885) →
StateProcessor.Process (core/state_processor.go:68) → block_validator
(state root + BAL hash vs locally re-derived BAL, block_validator.go:183)
→ VALID/INVALID` — and, for canonicalization,
ConsensusAPI.ForkchoiceUpdatedV4 (line 223). Full call-path diagram and
state/root/trace availability table: client/INSERTION-POINT.md.
```

The gate's operative position (the central architectural finding):

between `newPayload` (execution complete) and `forkchoiceUpdated` (canonicalization). Both implemented modes sit there:

- **Shadow mode: OPERATIONAL.** All scenarios/adversarial runs executed through the real Engine API against the unmodified client.
- **Consensus-visible SIMULATION: OPERATIONAL** (CL-side rule), explicitly NOT an Ethereum consensus change (§39 honored).

Real payload execution: REAL Geth payloads (built by Geth's miner from the real txpool; executed by Geth's state processor; BAL computed and execution-validated by Geth) — 100+ payloads across scenarios, adversarial, determinism, perf runs.

SRW3 policy: `srw3_policy.json` — source: deployed contract addresses + governance invariants; authority: deployment-pinned digest (`deployment.json`), policy CANNOT define its own authority (A-G7 preserved; H-A1 demonstrates rejection).

Execution evidence (all client-derived, provenance recorded field-by-field in `python/evidence.py`):

- State evidence: `payload.stateRoot == header.stateRoot` (client's own commitment; §9 adapter, client scheme NOT replaced).
- Effect evidence: ordered canonical trace (READ/WRITE/CALL/LOG) from receipts + `callTracer` + `prestateTracer(diffMode)`; BAL decoded independently from the payload.
- Authority evidence: AuthorityCertificate chain (level-0 protocol root → level-1 client → level-2 evidence), 1G NoCircularAuthority rules.

Required scenarios (§14)

Scenario	Ethereum	SRW3	Layer / note
H1 honest	VALID	SRW3_VALID	—
H2 aggregate-cap violation	VALID	SRW3_REJECT	invariant: INV-AGG-CAP (borrowCap read from client post-state — boundary finding)
H3 hidden write	VALID	SRW3_REJECT	effect-completeness: hidden=1 (declared ≠ executed)
H4 stale state/evidence	VALID	SRW3_REJECT	evidence-binding: stale parent root
H5 authority substitution	VALID	SRW3_REJECT	authority:authsrc
H6 policy substitution	VALID	SRW3_REJECT	policy-context
H7 execution-context substitution	VALID	SRW3_REJECT	policy-context (config digest)
H8 self-authorizing evidence	VALID	SRW3_REJECT	authority:authcircle

Scenario	Ethereum	SRW3	Layer / note
H9 cross-app interaction violation	VALID	SRW3_REJECT	interaction: IO-ORACLE-ORDER (oracle write AFTER borrow)
H10 no-policy path	VALID	DISABLED	non-invasive: block hashes identical with/without SRW3 (H10-D, below)

Every scenario runs against REAL execution: H2's cap breach, H3's hidden write, H9's ordering violation are executed by Geth and validated as Ethereum-VALID — the SRW3 layer is semantically ADDITIONAL (Question 3: YES, demonstrated).

Adversarial results (§30)

Case	Result
H-A1 malicious policy	fail-closed — governance pin rejects before evaluation (PolicyError)
H-A2 malicious authority certificate	SRW3_REJECT (authroot)
H-A3 valid-ETH/invalid-SRW3 invariant	SRW3_REJECT (invariant; = H2)
H-A4 hidden + phantom effect	SRW3_REJECT (effect-completeness; hidden=1, phantom=1)
H-A5 stale lineage	SRW3_REJECT (policy-context)
H-A6 branch/reorg replay of a REJECTED commitment	SRW3_REJECT — persists on its branch; duplicate newPayload accepted by client; re-evaluation deterministically rejects; no resurrection
H-A7 policy-version substitution	SRW3_REJECT (policy-context; = H6)
H-A8 client-identity substitution (rogue EL cert)	SRW3_REJECT (authsrc)
H-A9 execution-config substitution	SRW3_REJECT (policy-context; = H7)
H-A10 authority-source circularity (loop)	SRW3_REJECT (authsrc)
H-A11 missing SRW3 evidence	SRW3_ERROR — fail-closed in enforcing mode; recorded non-enforcing in shadow
H-A12 malformed (tampered) evidence	SRW3_REJECT (evidence-binding)
H-A13 SRW3 timeout	SRW3_ERROR — fail-closed; never auto-VALID
H-A14 client restart	SRW3_VALID — verdict + lineage record identical after restart+replay; head restored
H-A15 duplicate payload	lineage idempotent (no second record; client VALID/VALID)
H-A16 forkchoice transition	lineage follows forkchoice; per-branch records persist; head returns correctly after reorg-back

Reorg / restart / duplicate behavior (§21-§22)

- **Reorg:** SRW3 lineage is a client-local projection: records are keyed by block hash, the head follows forkchoiceUpdated; rejected commitments belong to their branch and reappear only if their branch becomes canonical again — in which case re-evaluation is deterministic (same verdict). Formally: SRW3 lineage head = the client's canonical head restricted to SRW3-recorded payloads.
- **Restart/replay:** records are crash-atomic files; re-processing a known payload is idempotent (identical record → no-op; divergent verdict

→ lineage inconsistency error, never silently overwritten).

- **Policy evaluation dependence:** evaluated at each payload's own fork context; nothing depends on finalized/safe heads in Phase 1H (recorded as an open design question for real deployment: §27 table).

Failure behavior (§24-§25)

FailurePolicy(SRW3): adapter/evidence failures produce verdict

SRW3_ERROR — a THIRD verdict class, never conflated with SRW3_VALID.

Shadow mode: recorded, non-enforcing (explicitly the only place where unavailability does not block commitments — the mode is non-enforcing by definition and by construction). Enforcing (consensus-visible-sim) mode: **FAIL-CLOSED** (ERROR ⇒ treated as INVALID). Justification: an unavailable gate must never become an automatic security-valid commitment (H8/H-A11/H-A13; verified).

BAL / access-list analysis (§19)

- **BAL write set == execution-derived write set** (application contracts; value-equal; EIP-7928 as implemented by Geth v1.17.7:
[slot, [[blockAccessIndex, postValue],...]]).
- **Write attribution EXISTS** (blockAccessIndex per change) — an EIP-7928-final refinement over the Phase 1G analysis.
- ****Reads remain unattributed keys without values; no call/return structure; no read observations.**** The interaction layer (H9) is NOT evaluable from BAL alone.
- **Unchanged invariant bounds are not in the BAL** (only touched slots); post-state reads (eth_getStorageAt at the payload hash) remain necessary — recorded as a boundary finding (§14 H2 depends on it).
- **BAL is execution-VALIDATED by the client:** Geth re-derives the BAL during import and requires equality with the header hash (block_validator.go:183-196); any tampering yields INVALID (demonstrated: tampered BAL → INVALID). A presented BAL therefore cannot hide or invent effects against the client — a STRENGTHENING of Mode H-B evidence relative to Phase 1G.
- **Phase 1G finding PRESERVED (and strengthened):** BAL ≠ complete SRW3 security-effect trace.

Performance (§18; real devnet path, no synthetic microbenchmarks)

Workload	txs	gas	evidence extraction (median)	gate (median)	evidence JSON	overhead vs produce+execute
small	1	17,775	7.6 ms	0.09 ms	6.2 KB	~0.5 %

Workload	txs	gas	evidence extraction (median)	gate (median)	evidence JSON	overhead vs produce+execute
medium	10	177,858	7.5 ms	0.09 ms	14.1 KB	~0.4 %
large	60	1,425,576	17.5 ms	0.13 ms	91.8 KB	~0.4 %

Adapter memory: +4.2 MB RSS over a full scenario suite. Extraction scales with trace volume (2 → 206 events); the gate itself is sub-millisecond at this scale. Client-behavior finding: Geth v1.17.7's payload builder was observed to never commit batches of 40+ heavily-conflicting SSTOREs into a payload (empty commits at any wait) — documented, and the large workload was reshaped to stay within verified bounds; getPayloadV6 was observed to DELIVER-AND-CLOSE a build job, so calling it before the worker's first commit seals an empty payload (harness accounts for both).

Determinism (§17)

Check	Result
independent fresh builds → identical block hashes (A1 vs A2)	TRUE
payload bytes identical (A1 vs A2)	TRUE
payload REPLAY on independent instance → identical chain, evidence digests, execution IDs, verdicts (A vs B)	TRUE / TRUE / TRUE / TRUE
H10-D: identical blocks with vs without SRW3	TRUE

Conditional clause (disclosed, not hidden): determinism is conditional on the canonical payload. The CL-side payload-build retry (a fresh build job requires a new timestamp) is an attribute degree of freedom that propagates into state (updatedAt) and hence into evidence digests — the experiment detects and excludes such runs (rebuilt flags), and the finding is itself a determinism hazard for any client-side gate.

Engine API compatibility (§18/§2)

- All experiments run on the CURRENT spec surface (Amsterdam: newPayloadV5/getPayloadV6/FCUv4/getPayloadBodiesByHashV2); no historical version was hard-coded; the method version follows the active fork.
- Bogota (newPayloadV6, inclusion lists) exists in the frozen spec but is beyond Geth v1.17.7's implemented surface — recorded as a client-side limitation.
- debug_chainConfig does not exist in v1.17.7 (NOT EXPOSED BY CLIENT; chain config via admin_nodeInfo.protocols.eth.config).
- Engine INVALID currently means EXECUTION-invalid. SRW3 rejection is NOT execution-invalid: mapping it onto INVALID overloads the semantics (see consensus-visible simulation, below).

Consensus-visible simulation (§15)

Step	Result
CS1 honest	SRW3_VALID → canonicalized normally
CS2 violating	client says VALID (+ latestValidHash=payload) — simulated rule translates SRW3_REJECT → INVALID: payload NOT canonicalized (rejectedBlockNotCanonical: true)
CS3 build on rejected	client builds and VALIDates a child of the SRW3-rejected payload — the client has no notion of SRW3-invalidity; only the CL-side rule prevents extension
CS4 re-presentation	client VALID (already imported); SRW3 verdict stable (REJECT); lineage record unchanged
CS5 recovery	honest payload → SRW3_VALID → canonicalized; chain continues

Consequences (analysis recorded in the transcript): latestValidHash semantics would be overloaded (a security-invalid block has an execution-valid ancestor chain); fork choice must treat the rejected head as nonexistent; any client WITHOUT the policy extends the rejected branch → a persistent fork — ****SRW3 enforcement at this boundary is a fork-choice/protocol rule, not an execution-validity rule.****

Consensus/protocol change boundary (§27)

Requirement	Client-local	Engine API extension	Consensus change
Receive SRW3 policy	YES (local config/pin)	—	—
Obtain execution evidence	YES (debug traces + BAL + storage reads)	nicer: trace/state-diff as Engine datum	—
Evaluate SRW3	YES (sidecar or in-process hook)	—	—
Store lineage	YES (sidecar)	optional: receipt/block metadata	—
Reject local security commit	YES (refuse to build/canonicalize locally)	—	—
Return Engine INVALID	NO (INVALID means execution-invalid)	YES (new status or new field, e.g. securityInvalid)	—
Make rejection fork-choice relevant	NO	YES (CL-visible rejection signal)	YES (all clients must honor it identically)
Make policy consensus-authoritative	NO	—	YES (policy digest in protocol/consensus data)
Make SRW3 mandatory	NO	—	YES (validity rule change + client interoperability)

Multi-client interoperability requirements (§26)

Not implemented (single client, per spec). Derived requirements for Geth/Nethermind/Reth/Besu to reach identical SRW3 results:

1. identical policy distribution (pinned digest, out-of-band);
2. identical effect representation: the ordered canonical trace must be derived identically — requires either (a) a standardized trace (EIP-7928 BAL + a read-attribution extension + call structure) or (b) per-client adapters pinned to identical semantics;
3. identical authority evidence: client identity + chain-config digest

must be reported uniformly (no standard today);

4. identical security result: the verdict function must be a single implementation (the frozen Gate) consuming the above — divergence in 1-3 forks the consensus-visible mode by construction.

What is proved / demonstrated / assumed / refuted / open

- **PROVED (frozen K, inherited, unmodified):** the Gate_G machinery — non-circular authority (AZG1-AZG5), record/policy binding, branch-wise verdict exactness (28 kprove claims on the phase1g baseline).
- **DEMONSTRATED ON REAL CLIENT:** integration at the Engine API boundary; Ethereum-valid $\nrightarrow$ SRW3-valid (H2/H3/H9); authority/policy/context substitution rejection; state-root binding; BAL execution-validation; restart/replay/reorg lineage behavior.
- **EXPERIMENTALLY VALIDATED:** determinism (§17 table); performance overhead ($\sim 0.4-0.5\%$); failure non-bypassability.
- **SPECIFICATION THEOREM:** H9 consensus-boundary characterization (§16).
- **ASSUMED:** A-G2 (protocol/consensus root is protocol-given — unchanged from 1G); A-G7 (policy authority = governance input); the adapter's own trusted-computing-base (Python harness) is NOT covered by any proof.
- **NOT BUILT (disclosed):** the in-process Geth hook (`client/patch/srw3-shadow-hook.patch`) — no Go toolchain in the environment; the demonstrated integration is the unmodified-client boundary path.
- **REFUTED (counterexample):** "BAL can replace the SRW3 effect trace" — read attribution/values and call structure are absent (§19); "SRW3 rejection fits existing Engine INVALID semantics" — it overloads them (§15).
- **OPEN / remains:** K-toolchain rebuild + mechanization of the two new boundary claims (H1-reduction, H6 shadow-safety); finalized/safe-head policy anchoring; multi-client interop (1-4 above); actual protocol integration (future phase).

Phase 1H authority-boundary conclusion

SRW3 embeds cleanly at the execution/consensus boundary of a real client today, in shadow mode, with real payloads, real evidence, real authority certificates, and a real policy — with measurable, small overhead and byte-identical determinism. The next arrow (mandatory enforcement) is a PROTOCOL problem: it requires (1) a validity/fork-choice rule that distinguishes security-invalidity from execution-invalidity, (2) consensus-authoritative policy distribution, and (3) cross-client

evidence interoperability. Each is precisely itemized in the §27 table. This is the intended successful Phase 1H result ("clean embedding, but mandatory consensus-visible enforcement requires a new protocol commitment rule").

Reproducibility (§33)

- client: `geth-linux-amd64-1.17.7-3d858f85` (official prebuilt; sha256 in `provenance/MANIFEST.txt`), source `go-ethereum v1.17.7 @ 3d858f85`
- Engine API spec: `execution-apis @ 3b9944a` (frozen in `spec-baseline/`)
- chain/fork configuration + genesis: `devnet/genesis.json`
- SRW3 policy: `policy/srw3_policy.json` (+ `deployment.json` pin)
- payload fixtures + traces + evidence + decisions: `fixtures/`, `transcripts/`
- harness: `python/` (stdlib + eth-account only), devnet scripts: `devnet/`
- patch: `client/patch/srw3-shadow-hook.patch` (NOT BUILT — disclosed)

Artifact index (§34)

- `model/MODEL.md` — boundary model
- `client/` — `CLIENT-SELECTION.md`, `INSERTION-POINT.md`, `patch/`, adapter config (`devnet/`)
- `policy/` — `srw3_policy.json` + `deployment.json` (governance pin)
- `formal/` — claim module + theorem classification
- `python/` — adapter, gate, lineage, harness, scenario/adversarial/determinism/perf/BAL runners
- `tests/` — unit/integration/devnet/adversarial (the runners are the test hierarchy: unit-level adapter checks, integration = Engine API interaction, devnet = full CL→EL→SRW3 flow, adversarial = H-A*)
- `fixtures/` — contracts, policies, lineage records, transcripts
- `transcripts/` — build, client, engine, srw3, adversarial, devnet, perf
- `report/` — this report (MD + PDF)
- `provenance/` — `PROVENANCE.md`, `MANIFEST.txt`, hashes
- `spec-baseline/` — frozen Execution API spec + hashes

Phase 1H completion verdict

****COMPLETED** — 21/23 completion criteria met directly, 2 met with disclosed qualifications (§35: criterion 21 "Python ↔ K ↔ client

consistency where applicable" = client↔Python verified byte-identically +
K layer classified/inherited rather than re-mechanized; criterion 32
end-to-end devnet = consensus client ROLE simulated by the harness against
the real execution client through the real Engine API, which IS the
CL→EL→SRW3 flow minus third-party consensus-client software).**
Verdict: **PARTIALLY PROVED / DEMONSTRATED** — integration proven on a
real client in shadow + simulated-enforcement modes; consensus-visible
enforcement REQUIRES CONSENSUS/PROTOCOL CHANGE (itemized), exactly as the
phase specification's success condition anticipated.